现代大语言模型推理中KV缓存的体系结构感知管理与层级化迁移策略研究报告

在大语言模型（LLM）从通用对话向复杂智能体（Agent）、图结构推理（Graph-based Reasoning）及长上下文工具利用演进的过程中，推理系统的性能瓶颈已发生根本性转移。早期的研究重点主要集中在模型权重的量化与计算算子的优化上，然而随着上下文窗口从早期的 4K、8K 迅速扩展至 128K 乃至 1M 以上，Key-Value（KV）缓存的显存占用呈现出严峻的线性增长态势[1]。在解码阶段，自注意力机制（Self-Attention）要求模型在生成每一个新 Token 时，都必须访问之前所有 Token 对应的键（Key）和值（Value）向量[3]。这意味着 KV 缓存不仅是显存容量的主要消耗者，其频繁的存取操作也使推理过程陷入严重的访存带宽受限状态[4]。

在资源受限的异构计算环境下，如何通过软件定义的方式，在不牺牲模型精度和推理吞吐的前提下，实现 KV 缓存的高效监控、动态迁移与碎片化管理，已成为构建高性能大模型推理引擎的核心考点。本报告将针对 KV 缓存的体系结构特性、访问模式特征、层级化存储管理及透明集成技术路径进行详尽的调研与深度分析。

KV 缓存的技术内幕与内存瓶颈量化分析

KV 缓存的引入是为了解决 Transformer 架构在自回归生成过程中的冗余计算问题。在标准 Transformer 层中，注意力分数的计算公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

在解码第 n 个 Token 时，如果不使用缓存，系统需要重新计算前 $n-1$ 个 Token 的 K 和 V 矩阵，导致总计算复杂度达到 $O(N^2)$[3]。通过缓存已计算的 K 和 V，每一步生成仅需计算当前 Token 的 Q, K, V 并与缓存拼接，将计算复杂度降低至 $O(N)$，但这是以空间换时间的策略，导致了内存需求的急剧膨胀[3]。

KV 缓存占用空间的量化评估

KV 缓存的存储规模由模型深度、头数、隐藏层维度以及数据精度共同决定。对于使用全量多头注意力（MHA）的模型，每个 Token 占用的显存字节数为：

$$\text{Size}_{\text{token}} = 2 \times \text{Layers} \times \text{Heads} \times \text{Dim}_{\text{head}} \times \text{Bytes}_{\text{element}}$$

以 Llama-2-7B 模型为例（32层，32个头，每头维度128），在 FP16 精度下，每个 Token 需占用约 512 KB 显存[5]。当上下文长度达到 128K 时，仅单条请求的 KV 缓存就需占用 64 GB，这已超过了大多数主流显卡（如 NVIDIA RTX 4090 或 A10G）的显存总量[4]。

模型架构指标	层数 (L)	KV头数 (H)	每头维度 (D)	FP16单Token占用	128K上下文容量
Llama-2-7B	32	32	128	512 KB	64.0 GB
Mistral-7B (GQA)	32	8	128	128 KB	16.0 GB
Llama-3-8B (GQA)	32	8	128	128 KB	16.0 GB
Llama-3-70B (GQA)	80	8	128	320 KB	40.0 GB
Qwen-2.5-72B	80	8	128	320 KB	40.0 GB
DeepSeek-V3 (MLA)	128	128	512*	~160 KB*	20.0 GB*

尽管分组查询注意力（GQA）和多头潜变量注意力（MLA）等技术通过减少 K, V 头数或引入低秩压缩，极大地缓解了单 Token 的开销，但在超长上下文场景下，显存墙依然是无法逾越的物理限制 [1]。特别是对于支持 1M 上下文的下一代模型，KV 缓存的规模将达到数百 GB，这要求推理引擎必须具备跨越显存、内存（RAM）乃至外存（SSD）的自动化分层管理能力 [1]。

推理阶段的算力与访存不对称性

推理过程分为预填充（Prefill）和解码（Decode）两个截然不同的阶段，这对 KV 缓存的管理提出了动态的需求 [3]：

1. 预填充阶段（**Prefill**）：模型并行处理输入 Prompt，其计算密集度高，受限于 GPU 的浮点运算性能（FLOPS）。此时会一次性生成大量的 KV 缓存并写入显存 [4]。

2. 解码阶段（**Decode**）：生成过程是顺序的，每生成一个新 Token 都需要读取模型权重和整个 KV 缓存。由于每个 Token 的计算量极小（访存比极低），这一阶段是典型的内存带宽受限（Memory-bound）[4]。

这种不对称性意味着，在解码长序列时，GPU 大量的时间被消耗在从显存中读取 KV 数据上，如果数据不在显存中，则需要跨越 PCIe 总线从内存或外存获取，巨大的带宽鸿沟（显存 TB/s 级别 vs 内存数十 GB/s 级别）将直接导致推理速度的断崖式下跌[4]。

KV 缓存访问模式的特征识别与热度定义机制

为了实现高效的迁移，系统必须具备对 KV 缓存访问模式的深度感知能力。不同于传统的 CPU 缓存行访问，KV 缓存在 Transformer 推理中的访问具有极强的先验结构[3]。

宏观层面的 U 型层级重要性分布

研究表明，大模型不同层对于 KV 缓存的依赖程度和敏感度并不均衡。在处理相同内容但前缀不同的任务时，不同层的 KV 向量展现出一种“U 型”的相似性分布[17]：

- 浅层（**Early Layers**）：主要负责基础特征提取，受前缀变化影响较小，KV 缓存的对齐度较高，具备较强的重用性[17]。
- 中间层（**Middle Layers**）：负责语义整合与上下文锚定，对前缀极其敏感。这里的 KV 缓存随输入变化剧烈，偏差最大，通常是推理过程中的“最关键区域”，应优先保留在高速显存中[17]。
- 深层（**Late Layers**）：逐渐收敛于输出分布预测，KV 偏差再次减小[17]。

这种层间差异暗示了一种“层级感知迁移策略”：系统可以根据当前推理的深度，动态调整显存中保留的 KV 块比例，甚至对浅层 KV 采取更积极的压缩或外迁策略，而对中间层实施强一致性保护[17]。

微观层面的 Token 访问异质性

在同一层内，不同的 Token 对注意力分数的贡献也存在巨大差异，这直接定义了 KV 块的“热度”[13]：

1. 注意力汇聚现象（**Attention Sinks**）：模型往往会对序列中的前几个 Token（特别是 <BOS> Token）分配极高的注意力权重，即使这些 Token 在语义上并不重要[13]。这些“汇聚 Token”必须永久驻留在显存中，否则会导致模型崩溃。
2. 局部局部性（**Local Recency**）：最近生成的 Token 通常在后续生成中被频繁引用，其访问频率呈现时间局部性[13]。
3. 语义锚点（**Semantic Anchors**）：对于智能体或长文档任务，特定的关键词（如变量定义、关键指令或实体名）会被跨时步频繁访问，这些 Token 所在的 KV 块即为“热块”[22]。

基于多维指标的热度定义模型

为了量化热度，本系统可以定义一个动态权重模型。设 KV 块为 B_i，其在时间步 t 的热度值

$H(B_i, t)$ 可表示为：

$$H(B_i, t) = w_a \cdot \bar{A}_i + w_f \cdot F_i + w_r \cdot \frac{1}{\Delta t_i} + w_l \cdot \text{Layer}(B_i)$$

其中：

- $\bar{A}_i$ 为块内 Token 的平均注意力权重历史平均值[20]。
- F_i 为该块在最近 N 个时间步内的访问频率。
- Δt_i 为该块距离上一次被访问的时间间隔。
- $\text{Layer}(B_i)$ 是基于 U 型分布的层重要性加权系数[17]。

通过实时维护这样一个热度表，系统可以精确识别出哪些 KV 块属于当前生成步骤的“热数据”，哪些是偶尔访问的“温数据”，哪些是长期未触碰的“冷数据”[16]。

动态 KV 缓存迁移引擎：跨存储介质的数据流转

针对 KV 缓存的线性增长，必须构建一套能够掩盖数据传输延迟的迁移引擎，在 HBM、System RAM 和 NVMe SSD 之间实现无缝调度[10]。

层级化存储系统的带宽与时延约束

存储层级	物理位置	带宽 (理论/实测)	典型访问延迟	管理策略建议
Tier 1: HBM	GPU 显存	2.0-3.5 TB/s	< 1 µs	存放高热度数据 (Sink, 局部窗口)
Tier 2: RAM	CPU 内存	32-64 GB/s	100-500 ns	存放温数据 (最近使用的历史上下文)
Tier 3: SSD	本地 NVMe	3.5-7.0 GB/s	10-100 µs	存放冷数据 (长序列背景资料)

Tier 4: Remote	对象存储	1.0-10 GB/s	10-100 ms	跨请求共享的持久化前缀缓存

跨层迁移的核心挑战在于 PCIe 总线的带宽限制 [4]。一次典型的 16K Token 的 KV 缓存迁移（FP16 下约 8GB）通过 PCIe 4.0 x16 约需 0.3-0.5 秒，这对于单 Token 生成时间要求在 50ms 以内的实时推理场景是完全不可接受的 [9]。

迁移引擎的关键技术实现

1. 异步流水线（**Asynchronous Pipelining**）：系统利用 cudaMemcpyAsync 或 PTX 异步指令（如 cp.async.bulk）将数据迁移与 GPU 计算核函数的执行重叠 [16]。在计算当前层的注意力时，预先将下一层可能用到的 KV 块从 CPU RAM 迁移回显存 [16]。
2. 关键块预测与预取（**Predictive Prefetching**）：如 KVSwap 系统所示，由于完整 Key 向量庞大，可以在内存中保留一个高度压缩的 Key 表示（如低秩投影或 4-bit 量化版本）[16]。每步推理前，先用 Query 对压缩 Key 进行快速相似度扫描，预测哪些磁盘块将被高权重访问，提前触发异步读取请求 [16]。
3. 零拷贝与直连存储（**Zero-copy & GDS**）：通过 NVIDIA GPUDirect Storage (GDS) 或 AMD 类似的直连技术，绕过 CPU 的内存拷贝过程，将 NVMe SSD 中的数据直接通过 PCIe 传输到 GPU HBM [11]。这不仅减少了 CPU 的占用，更极大提升了“冷数据”转“热”的转换效率。
4. 迁移过程中的动态压缩：在将数据由显存移往显存之外时，系统可以实时执行低比特量化（如将 FP16 压缩为 INT4）[4]。这虽然会消耗极少量的计算资源，但能使传输数据量减半甚至更多，从而变相提升了总线带宽的有效吞吐量 [14]。

KV 缓存存储分配器：优化内存碎片与并发访问

随着推理进程的推进，KV 缓存的动态增长极易导致内存碎片，这在长上下文并发请求下会导致 GPU 利用率的急剧下降 [8]。

从连续分配到 PagedAttention 的范式转移

传统推理框架采用预分配连续空间的做法，为了支持最大长度（如 2048），必须为每个请求保留完整空间，这导致了严重的内存浪费（内部碎片和预分配浪费高达 60-80%）[8]。

PagedAttention（vLLM）引入了类似虚拟内存的分页机制 [4]：

- 块池管理（**Block Pool**）：KV 缓存被切分为固定大小的物理块（Logical KV Blocks，通常每块 16 或 32 个 Token）[8]。
- 映射表映射（**Block Table**）：每个请求维护一张逻辑块到物理块的映射表。不同物理块可以散落在显存各处，从而消除了外部碎片 [8]。
- 内容寻址与共享（**Prefix Caching**）：通过哈希算法标识前缀块，多个具有相同 System Prompt 的请求可以物理上共享同一份物理块，从而在内存中几乎“免费”地支持大规模并发

请求[4]。

针对分层存储的自定义分配器设计

对于涉及外存迁移的系统，分配器需要扩展其职责：

- 跨设备统一编址：实现一套统一的逻辑地址空间，屏蔽 HBM、RAM、SSD 的物理差异。当分配器发现 HBM 空间不足时，自动触发 LRU 迁移，并将物理块标记为“Offloaded”状态[10]。
- 碎片规避策略：借鉴 mimalloc 或 jemalloc 的分级分配（Size Classes）思路，将 KV 块分配在对齐的页边界上，减少因元数据操作导致的 CPU 锁竞争[41]。在异构计算环境下，分配器需感知 NUMA 节点，确保 CPU 端的 KV 备份位于距离 GPU 最近的内存控制器所管辖的区域[43]。
- 动态扩缩容：随着序列增长，分配器应支持“渐进式分配”。例如，初始分配一个小块，当接近满额时再原子化地链接下一个物理块，避免一次性预分配超长序列带来的显存压力[8]。

透明集成路径：无需修改推理代码的系统钩子

赛题要求系统具备透明集成的能力，即无需修改 vLLM、HuggingFace 等框架的源代码。这通常通过符号拦截（Interception）和虚拟化驱动技术实现[44]。

LD_PRELOAD 与 CUDA 符号包装

通过在程序启动前设置 LD_PRELOAD 环境变量，我们可以强制加载自定义的共享库（如 libkvcache_manager.so），并重写关键的底层函数[41]：

- 内存分配拦截：包装 cudaMalloc、cudaMallocManaged 或驱动层的 cuMemAlloc[45]。系统可以在此环节接管地址管理，为每个分配请求返回一个受管辖的虚拟地址，并实时记录该内存区域的初始归属（即是否为 KV 缓存区域）[45]。
- 核函数监控：通过包装 cudaLaunchKernel 或底层的 cuLaunchKernel，系统可以获取当前正在执行的任务信息。结合核函数的参数（引用的内存地址），可以实时分析出哪些 KV 块正在被计算核心访问，从而更新其热度权重[44]。
- 异步迁移触发：在拦截到 cudaStreamSynchronize 或类似同步调用时，系统可以检查是否有预取计划需要执行，或者是否有低热度数据需要被换出显存[31]。

虚拟内存管理 (VMM) 的深度应用

NVIDIA 自 CUDA 10.2 起引入了 VMM API（如 cuMemCreate 和 cuMemMap），这为透明迁移提供了极致的技术底座[45]：

1. 地址空间预留：使用 cuMemAddressReserve 申请一个足以容纳模型全量 KV 缓存的巨大连续虚拟地址空间（无需物理显存支持）[45]。
2. 物理绑定：仅当真正需要数据时，才调用 cuMemCreate 创建物理内存句柄，并使用 cuMemMap 将其绑定到预留的虚拟地址上[45]。
3. 无感迁移：当需要将块从显存移到内存时，可以在后台将物理句柄解除绑定，移动数据到内存备份，并释放物理显存，而应用层看到的指针地址完全不变[45]。当应用下次访问该地址时

，通过页缺失拦截或预取机制重新绑定物理显存。

硬件兼容性与国产异构计算平台适配

随着国产算力的崛起，系统必须能够同时支持 NVIDIA/AMD GPU 以及华为昇腾（Ascend）、寒武纪（Cambricon）等国产硬件平台 [1]。

华为昇腾 (Ascend) 与 CANN 架构的特性适配

华为昇腾 AI 处理器在 KV 缓存管理方面有其独特的设计优势 [1]：

- **HCCS 与高速互联**：昇腾 910C 等芯片通过华为高速互联系统（HCCS）提供极高的芯片间带宽（实测可达 2 TB/s 以上），这使得在多卡 SuperPoD 环境下，KV 缓存的跨卡迁移速度甚至快于单卡内的 PCIe 迁移 [52]。
- **PDC 预填充-解码-缓存解耦架构**：昇腾 CloudMatrix-Infer 方案提出了将 Prefill、Decode 和 Caching 彻底分离开的架构 [1]。其中专用的缓存节点（Caching Nodes）通过超高带宽的统一总线（Unified Bus, UB）为解码节点提供海量的 KV 检索能力，这实际上在硬件层面实现了一个分布式的 KV 缓存分层系统 [1]。
- **CANN 算子融合与并行**：在国产硬件上，我们可以利用 CANN 的 HCCL 库进行高效的集体通信，实现在 KV 缓存迁移过程中的异步数据分发 [43]。

硬件架构对比与互联协议

硬件平台	核心互联协议	**KV**迁移瓶颈点	特色优化工具
NVIDIA	NVLink / NVSwitch	PCIe 带宽 / 显存交换延迟	LMCache, vLLM, TensorRT-LLM
AMD	Infinity Fabric	访存延迟 / 异步拷贝掩盖	ROCm, vLLM-AMD
Huawei	HCCS / UB-Mesh	集体通信开销	CANN, MindSpore, CloudMatrix
Intel	Xe Link	计算/迁移资源竞争	IPEX, vLLM-CPU

对于国产硬件，系统需要适配 torch_npu 或华为原生 CANN API，利用其提供的虚拟内存管理接口（类似于 CUDA 的 VMM）来实现透明的跨层迁移 [43]。

性能评估指标体系与服务质量 (SLO)

为了量化系统的优化效果，必须建立多维度的评估模型 [7]。

核心评估维度

1. **KV 缓存命中率（Cache Hit Rate）**：衡量迁移策略的准确性。理想状态下，当前生成步骤所需的 KV 块应有 95% 以上驻留在显存中 [40]。
2. 平均访问延迟（**Average Access Latency**）：包含迁移产生的额外开销。目标是使显存缺失带来的延迟增加控制在单 Token 生成时间的 5% 以内 [7]。
3. 有效上下文扩展倍数：在相同物理显存下，系统能稳定支持的最大序列长度提升。通常目标为 4x 到 10x 的扩展 [6]。
4. 吞吐率提升（**Throughput Improvement**）：由于显存释放带来的并发数增加，单位时间内处理的总 Token 数应有显著提升（通常可达 2-4 倍）[8]。

系统开销监控

迁移算法本身不能成为新的瓶颈。评审中需重点考察：

- **CPU 占用率**：监控与预测算法在后台运行时的 CPU 负载，不应影响推理引擎的控制逻辑。
- 元数据开销：维护数百万个 KV 物理块的状态所需的额外内存消耗。
- 确定性与抖动：迁移过程是否会导致 ITL（Inter-Token Latency）的显著抖动，影响用户体验 [7]。

结论与技术演进展望

本赛题的研究不仅关乎工程实现，更触及了现代 LLM 推理系统的核心矛盾——有限的显存容量与近乎无限的生成需求之间的对立。通过对 KV 缓存访问模式的深度理解、分层存储体系的精细调度以及透明系统钩子的巧妙应用，我们可以极大突破单卡的物理限制。

未来，KV 缓存管理将朝着“极致压缩”与“全栈联邦”方向演进：一方面，像 MLA 和 TurboQuant 这样的极致压缩技术将进一步减少迁移的数据量 [1]；另一方面，跨节点的分布式 KV 缓存池（如华为 CloudMatrix 愿景）将使大模型推理真正具备“无限记忆”的能力 [1]。对于开发者而言，构建一个能够感知模型结构（U 型分布）、感知输入特性（热度预测）且具备异构硬件自适应能力的系统，是提升大模型服务性价比、推动长序列应用落地的必然选择。

Works cited

1. Serving Large Language Models on Huawei CloudMatrix384 - arXiv, accessed April 13, 2026, https://arxiv.org/html/2506.12708v3
2. MoSKA: Mixture of Shared KV Attention for Efficient Long-Sequence LLM

Inference - arXiv, accessed April 13, 2026, https://arxiv.org/html/2511.06010v1
3. Comparative Characterization of KV Cache Management Strategies for LLM Inference, accessed April 13, 2026, https://arxiv.org/html/2604.05012v1
4. The KV Cache: The Hidden Memory Monster That Controls Your LLM's Speed - Medium, accessed April 13, 2026, https://medium.com/@rajan.sethi36/the-kv-cache-the-hidden-memory-monster-that-controls-your-llms-speed-4bb35b937396
5. LLM Inference: KV Cache and Optimization | by Tushar Vatsa - Towards AI, accessed April 13, 2026, https://pub.towardsai.net/llm-inference-kv-cache-and-optimization-39650deee77c
6. PackKV: Reducing KV Cache Memory Footprint through LLM-Aware Lossy Compression, accessed April 13, 2026, https://arxiv.org/html/2512.24449v2
7. LLM Inference Benchmarking - Measure What Matters | DigitalOcean, accessed April 13, 2026, https://www.digitalocean.com/blog/llm-inference-benchmarking
8. PagedAttention - vLLM - Mintlify, accessed April 13, 2026, https://www.mintlify.com/vllm-project/vllm/concepts/paged-attention
9. A Deep Dive into LLM Inference Latencies - Hathora Blog, accessed April 13, 2026, https://blog.hathora.dev/a-deep-dive-into-llm-inference-latencies/
10. Boosting LLM Performance with Tiered KV Cache on Google Kubernetes Engine, accessed April 13, 2026, https://cloud.google.com/blog/topics/developers-practitioners/boosting-llm-performance-with-tiered-kv-cache-on-google-kubernetes-engine
11. Architecting for Reuse: A Deep Journey into the Heart of KV Caching, accessed April 13, 2026, https://blog.purestorage.com/purely-technical/cut-llm-inference-costs-with-kv-caching/
12. How LLM Inference Works - Arpit Bhayani, accessed April 13, 2026, https://arpitbhayani.me/blogs/how-llm-inference-works/
13. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference | Request PDF - ResearchGate, accessed April 13, 2026, https://www.researchgate.net/publication/384210052_PyramidInfer_Pyramid_KV_Cache_Compression_for_High-throughput_LLM_Inference
14. Persistent Q4 KV Cache for Multi-Agent LLM Inference on Edge Devices - arXiv, accessed April 13, 2026, https://arxiv.org/html/2603.04428v1
15. Nvidia CUDA memory sharing - how does it work and when it's bad and when it's not?, accessed April 13, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1embb2c/nvidia_cuda_memory_sharing_how_does_it_work_and/
16. KVSwap: Disk-aware KV Cache Offloading for Long-Context On-device Inference - arXiv, accessed April 13, 2026, https://arxiv.org/html/2511.11907v1
17. RelayCaching: Accelerating LLM Collaboration via Decoding KV Cache Reuse - arXiv, accessed April 13, 2026, https://arxiv.org/html/2603.13289v1
18. Preprint: Toward Robust and Efficient ML-Based GPU Caching for Modern Inference - arXiv, accessed April 13, 2026, https://arxiv.org/html/2509.20979v1

19. A Closer Look into LLMs for Table Understanding - arXiv, accessed April 13, 2026, https://arxiv.org/html/2603.15402v1
20. (PDF) BaKlaVa -- Budgeted Allocation of KV cache for Long-context Inference, accessed April 13, 2026, https://www.researchgate.net/publication/389168051_BaKlaVa_--_Budgeted_Allocation_of_KV_cache_for_Long-context_Inference
21. SinkTrack: Attention Sink based Context Anchoring for Large Language Models, accessed April 13, 2026, https://openreview.net/forum?id=Gg1aPETCL6
22. Memory, Salience, and Why Your LLM Forgets: An Investigation We Actually Need to Run, accessed April 13, 2026, https://medium.com/@mbonsign/memory-salience-and-why-your-llm-forgets-an-investigation-we-actually-need-to-run-c7124b742319
23. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference - USENIX, accessed April 13, 2026, https://www.usenix.org/system/files/fast25-chen-weijian-impress.pdf
24. [RFC]: KVComp Sparse Attention for Long-Context Inference · Issue #5507 · vllm-project/vllm-ascend - GitHub, accessed April 13, 2026, https://github.com/vllm-project/vllm-ascend/issues/5507
25. Adaptive Multi-Objective Tiered Storage Configuration for KV Cache in LLM Service - arXiv, accessed April 13, 2026, https://arxiv.org/html/2603.08739v1
26. KVSwap: Disk-aware KV Cache Offloading for Long-Context On-device Inference - arXiv, accessed April 13, 2026, https://arxiv.org/pdf/2511.11907
27. llm-d/guides/tiered-prefix-cache/storage/README.md at main - GitHub, accessed April 13, 2026, https://github.com/llm-d/llm-d/blob/main/guides/tiered-prefix-cache/storage/README.md
28. KV Caching with vLLM, LMCache, and Ceph, accessed April 13, 2026, https://ceph.io/en/news/blog/2025/vllm-kv-caching/
29. The Architectural Paradigm of Multi-Adapter Inference: A Technical Analysis of LoRAX, accessed April 13, 2026, https://pub.towardsai.net/the-architectural-paradigm-of-multi-adapter-inference-a-technical-analysis-of-lorax-567c2f4851f0
30. xDeepServe: Model-as-a-Service on Huawei CloudMatrix384 - arXiv, accessed April 13, 2026, https://arxiv.org/html/2508.02520v4
31. Asynchronous KV Cache Prefetching - Emergent Mind, accessed April 13, 2026, https://www.emergentmind.com/topics/asynchronous-kv-cache-prefetching
32. Accelerating LLM Inference Throughput via Asynchronous KV Cache Prefetching | Proceedings of the AAAI Conference on Artificial Intelligence, accessed April 13, 2026, https://ojs.aaai.org/index.php/AAAI/article/view/39224
33. LMCache/LMCache: Supercharge Your LLM with the Fastest KV Cache Layer - GitHub, accessed April 13, 2026, https://github.com/lmcache/lmcache
34. vLLM Archives | LMCache Blog, accessed April 13, 2026, https://blog.lmcache.ai/en/tag/vllm-en/
35. Huawei to open-source its UB-Mesh data center-scale interconnect soon, details technical aspects — one interconnect to rule them all is designed to replace

everything from PCIe to TCP/IP | Tom's Hardware, accessed April 13, 2026, https://www.tomshardware.com/tech-industry/artificial-intelligence/huawei-to-open-source-its-ub-mesh-data-center-scale-interconnect-soon-details-technical-aspects-one-interconnect-to-rule-them-all-is-designed-to-replace-everything-from-pcie-to-tcp-ip
36. TurboQuant: What Developers Need to Know About Google's KV Cache Compression, accessed April 13, 2026, https://dev.to/arshtechpro/turboquant-what-developers-need-to-know-about-googles-kv-cache-compression-eeg
37. How PagedAttention resolves memory waste of LLM systems - Red Hat Developer, accessed April 13, 2026, https://developers.redhat.com/articles/2025/07/24/how-pagedattention-resolves-memory-waste-llm-systems
38. The Architecture Behind vLLM: How PagedAttention Improves Memory Utilization - Medium, accessed April 13, 2026, https://medium.com/@mandeep0405/the-architecture-behind-vllm-how-pagedattention-improves-memory-utilization-2f9b25272110
39. KV Cache Management - Nano-vLLM - Mintlify, accessed April 13, 2026, https://www.mintlify.com/GeeeekExplorer/nano-vllm/architecture/kv-cache
40. Implementation — vLLM, accessed April 13, 2026, https://docs.vllm.ai/en/v0.6.1/automatic_prefix_caching/details.html
41. mimalloc is a compact general purpose allocator with excellent performance. - GitHub, accessed April 13, 2026, https://github.com/microsoft/mimalloc
42. libmalloc, jemalloc, tcmalloc, mimalloc - Exploring Different Memory Allocators, accessed April 13, 2026, https://dev.to/frosnerd/libmalloc-jemalloc-tcmalloc-mimalloc-exploring-different-memory-allocators-4lp3
43. Optimization and Tuning — vllm-ascend, accessed April 13, 2026, https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/performance_and_debug/optimization_and_tuning.html
44. Revati: Transparent GPU-Free Time-Warp Emulation for LLM Serving - arXiv, accessed April 13, 2026, https://arxiv.org/html/2601.00397v1
45. Foundry: Template-Based CUDA Graph Context Materialization for Fast LLM Serving Cold Start - arXiv, accessed April 13, 2026, https://arxiv.org/html/2604.06664v1
46. CUDATracePreload is a dynamic tracing tool for CUDA and NCCL API calls. - GitHub, accessed April 13, 2026, https://github.com/vladandrew/CUDATracePreload
47. vLLM-MindSpore Plugin - Gitee, accessed April 13, 2026, https://gitee.com/mindspore/vllm-mindspore/blob/master/README_en.md
48. CRIUgpu: Transparent Checkpointing of GPU-Accelerated Workloads - arXiv, accessed April 13, 2026, https://arxiv.org/html/2502.16631v1
49. Heaps do lie: debugging a memory leak in vLLM. - Mistral AI, accessed April 13, 2026, https://mistral.ai/fr/news/debugging-memory-leak-in-vllm
50. Transparent inter-GPU memory migration - NVIDIA Developer Forums, accessed

April 13, 2026, https://forums.developer.nvidia.com/t/transparent-inter-gpu-memory-migration/275985
51. GPU Checkpoint/Restore Made Fast and Lightweight - USENIX, accessed April 13, 2026, https://www.usenix.org/system/files/fast26-zeng.pdf
52. AI Race: Can Huawei Close The AI Gap? - Forrester, accessed April 13, 2026, https://www.forrester.com/blogs/ai-race-can-huawei-close-the-ai-gap/
53. Huawei's New NVLink Competitor | ml-news – Weights & Biases - Wandb, accessed April 13, 2026, https://wandb.ai/byyoung3/ml-news/reports/Huawei-s-New-NVLink-Competitor--VmlldzoxNDQ1MjMyMg
54. Comparison Between NVIDIA CUDA and Ascend CANN. - ResearchGate, accessed April 13, 2026, https://www.researchgate.net/figure/Comparison-Between-NVIDIA-CUDA-and-Ascend-CANN_tbl2_395527742
55. Is it harder for Huawei's CANN to replace Nvidia's CUDA than for Hongmeng to replace Android? - EEWorld, accessed April 13, 2026, https://en.eeworld.com.cn/mp/XSY/a380171.jspx
56. Analysis: Huawei's Datacenter Chip Roadmap - A Strategic Pivot Toward Transparency and Scale - TechInsights, accessed April 13, 2026, https://library.techinsights.com/analysis-view/ARI-2604-803
57. Serving Large Language Models on Huawei CloudMatrix384 - arXiv, accessed April 13, 2026, https://arxiv.org/html/2506.12708v1
58. Groundbreaking SuperPoD Interconnect: Leading a New Paradigm for AI Infrastructure, accessed April 13, 2026, https://www.huawei.com/en/news/2025/9/hc-xu-keynote-speech
59. Serving Large Language Models on Huawei CloudMatrix384 - arXiv, accessed April 13, 2026, https://arxiv.org/abs/2506.12708
60. Quickstart-vLLM-Ascend — Unified Cache Manager - Read the Docs, accessed April 13, 2026, https://ucm.readthedocs.io/en/latest/getting-started/quickstart_vllm_ascend.html
61. LLM Inference Performance Benchmarking from Scratch - Phillippe Siclait, accessed April 13, 2026, https://phillippe.siclait.com/blog/llm-benchmarking-from-scratch
62. LLM Inference benchmarking guide — AWS Neuron Documentation - Read the Docs, accessed April 13, 2026, https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/nxd-inference/developer_guides/llm-inference-benchmarking-guide.html
63. Master KV cache aware routing with llm-d for efficient AI inference - Red Hat Developer, accessed April 13, 2026, https://developers.redhat.com/articles/2025/10/07/master-kv-cache-aware-routing-llm-d-efficient-ai-inference
64. KV-Cache Indexer: Architecture - GitHub, accessed April 13, 2026, https://github.com/llm-d/llm-d-kv-cache-manager/blob/main/docs/architecture.md

65. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference - arXiv, accessed April 13, 2026, https://arxiv.org/html/2510.09665v2